

Parcial 2 — Componente Práctico

Minería de Datos — Universidad del Norte

2025-1

v1.0

Nombre: _____

Código: _____

Instrucciones generales

En este parcial implementarán paso a paso un pipeline de **Locality-Sensitive Hashing (LSH)** para encontrar usuarios similares en el dataset **MovieLens**. Inicialmente trabajarán con el archivo **ml-latest-small.zip** (100 836 ratings, 610 usuarios, 9 724 películas) para desarrollar y validar el pipeline. Luego lo aplicarán a datasets más grandes (**ml-1m** y **ml-32m**) para evaluar la escalabilidad.

- La entrega debe ser un **notebook de Google Colab** con todas las celdas ejecutadas y celdas adicionales con las respuestas donde sea necesario.
- Las preguntas marcadas con **Responda:** deben contestarse en celdas Markdown o como **print** en celdas de código. Las respuestas deben ser claras y visibles.
- Se evalúa tanto el código como la interpretación de resultados.
- **No** se permite el uso de LLMs (ChatGPT, Copilot, Claude, Gemini, etc.) de ningún tipo durante el examen.
- Utilice `tqdm` en los loops principales de cada función implementada para monitorear el progreso y estimar la eficiencia de las implementaciones.

Datasets (descarga directa):

- **ml-latest-small** — <https://files.grouplens.org/datasets/movielens/ml-latest-small.zip>
- **ml-1m** — <https://files.grouplens.org/datasets/movielens/ml-1m.zip>
- **ml-32m** — <https://files.grouplens.org/datasets/movielens/ml-32m.zip>

1. Carga y exploración de datos (10 pts)

1.1. Cargue el archivo `ratings.csv` del dataset **ml-latest-small** usando **pandas**. Muestre:

- Las primeras 5 filas del `DataFrame`.
- El número de usuarios únicos, películas únicas y total de ratings.

1.2. Para cada usuario, construya el **conjunto** de películas que ha visto (ignorando el rating). Use `groupby` y `apply(set)`.

Grafique un histograma de la cantidad de películas por usuario. Incluya líneas verticales para los percentiles P50, P90 y P95.

Responda: ¿Qué tipo de distribución observa (simétrica, sesgada, etc.)? ¿Qué relación hay entre la media y la mediana? ¿Qué implicaciones tiene esto para medir similitud entre usuarios?

1.3.

2. Similitud de Jaccard exacta (20 pts)

2.1. Implemente la función de similitud de Jaccard:

```
def jaccard(set_a, set_b):
    """Retorna la similitud de Jaccard entre dos conjuntos."""
    # Su código aquí
```

- 2.2. Calcule la similitud de Jaccard para **todos** los pares de usuarios. Use `itertools.combinations` para recorrer los $\binom{610}{2} = 185,745$ pares. Almacene los resultados en una matriz simétrica (puede usar un `numpy.ndarray` o un `pd.DataFrame`).

Pista

Recuerde poner 1.0 en la diagonal (cada usuario es idéntico a sí mismo).

- 2.3. Grafique la distribución de todas las similitudes (triángulo superior de la matriz, sin la diagonal). Incluya percentiles P25, P50, P75, P90, P95 y P99.

Responda:

- ¿Cuál es la media y la mediana de las similitudes?
- ¿Qué porcentaje de pares tiene similitud exactamente 0?
- ¿Cuál es el valor del percentil P95? Este será nuestro **umbral** t para las secciones siguientes.

3. MinHash: estimación de Jaccard (20 pts)

La idea central de MinHash es que si aplicamos una permutación aleatoria π al universo de ítems, entonces:

$$\Pr[\min \pi(A) = \min \pi(B)] = J(A, B)$$

En la práctica, simulamos permutaciones con funciones hash de la forma $h(x) = (a \cdot x + b) \bmod p$, donde p es primo y a, b son aleatorios.

- 3.1. Implemente la función `lsh_params` que, dado un umbral t y filas por banda r , calcula el número de bandas b y el total de funciones hash m :

```
import math

def lsh_params(t, r):
    """Calcula b y m a partir del umbral t y filas por banda r."""
    b = math.ceil((1 / t) ** r)
    m = b * r
    return b, m
```

Usando el umbral t que encontró en la sección anterior (P95), genere una tabla con los valores de b y m para $r = 2, 3, 4, 5, 6, 7, 8$.

- 3.2. Implemente la función `build_signatures` que construye la matriz de firmas MinHash:

```
from sympy import nextprime
import numpy as np

def build_signatures(user_sets, m, seed=42):
    """
```

Construye la matriz de firmas MinHash.

Parametros

user_sets : list of sets

m : numero de funciones hash

Retorna

sig : np.ndarray de forma (m, n_users)

"""

Su codigo aqui

Pista

1. Obtenga el universo de todos los ítems e indexe cada ítem como un entero de 0 a $N - 1$. Encuentre el primo mas pequeño $p \geq N$ (`nextprime`) para usarlo como módulo de las funciones hash.
2. Genere m pares (a, b) aleatorios recordando los rangos para cada uno con `np.random.RandomState(seed)`.
3. Para cada función hash h_k , calcule $h_k(x) = (a_k x + b_k) \bmod p$ para cada ítem x en el conjunto de cada usuario.
4. La firma del usuario j en la fila k es $\text{sig}[k, j] = \min_{x \in S_j} h_k(x)$.

Construya las firmas para los 610 usuarios. Imprima la forma de la matriz resultante.

3.3. Implemente la función que estima Jaccard a partir de las firmas:

```
def minhash_sim(sig, i, j):
    """Similitud estimada = fraccion de filas donde sig[:,i] == sig[:,j]
    """
    return np.mean(sig[:, i] == sig[:, j])
```

Para **todos** los pares, calcule la similitud estimada por MinHash y compárela con la real (de la sección 2). Calcule:

- El **error absoluto**: $|\text{estimada} - \text{real}|$
- El **error relativo porcentual** (solo para pares con $J \geq t$): $\frac{|\text{estimada} - \text{real}|}{\text{real}} \%$

Grafique un histograma de cada uno.

Responda:

- ¿Cuál es el error relativo medio y en el P99 para los pares con Jaccard $\geq t$?
- El error absoluto teórico es $1/\sqrt{m}$. ¿El observado es mayor o menor? ¿Por qué?
- ¿Por qué el error relativo no tiene sentido cuando $J \approx 0$?

3.4. La curva S describe la probabilidad de que un par con similitud s sea identificado como candidato:

$$P(s) = 1 - (1 - s^r)^b$$

Grafique la curva S para $r \in \{2, 3, 4\}$, todos con el mismo umbral t . Marque con una línea vertical el umbral t .

Responda: ¿Qué efecto tiene aumentar r sobre la pendiente de la curva S en el umbral t ? Recuerde que $b = \lceil (1/t)^r \rceil$ y $m = b \cdot r$. ¿Cuál es el costo en términos de m al aumentar r ?

4. LSH: encontrar candidatos (20 pts)

4.1. Implemente la función `lsh_candidates` que encuentra pares candidatos usando bandas:

```
from collections import defaultdict

def lsh_candidates(sig, b, r):
    """
    Divide la firma en b bandas de r filas.
    Para cada banda, agrupa usuarios por su sub-firma
    (como tupla) en un diccionario (bucket).
    Pares en el mismo bucket son candidatos.

    Retorna
    -----
    candidates : set of tuples (i, j) con i < j
    """
    # Su código aquí
```

Ejecute LSH con los parámetros b y r calculados. Imprima:

- Número de pares candidatos.
- Fracción del total de pares que representan.

4.2. Implemente la función `evaluate_lsh` que evalúa la calidad de los candidatos:

```
def evaluate_lsh(candidates, sim_matrix, threshold):
    """
    Calcula precision y recall de LSH contra la verdad
    (sim_matrix >= threshold).

    Retorna
    -----
    dict con: true_similar, candidates, true_positives,
              precision, recall
    """
    # Su código aquí
```

Evalúe los candidatos contra la matriz de Jaccard exacta. Imprima precisión y recall.

4.3. Grafique la distribución de similitudes reales de los candidatos LSH superpuesta sobre la distribución de todos los pares. Marque el umbral t con una línea vertical.

Responda:

- ¿La mayoría de los candidatos están por encima o por debajo del umbral?
- ¿Qué significa un recall menor a 100%? ¿Qué tipo de pares “se pierden”?
- Compare los resultados con $r = 2$ vs. $r = 3$: ¿cuál tiene mejor precisión? ¿Cuál tiene mejor recall? ¿Por qué?

5. Análisis de resultados sobre ml-latest-small (10 pts)

- 5.1. Muestre los **10 pares de usuarios más similares** encontrados por LSH. Para cada par imprima: ID usuario A, ID usuario B, Jaccard real y estimación MinHash.
- 5.2. Compare los resultados de $r = 2$ y $r = 3$ en una tabla con columnas: r , b , m , umbral t , candidatos, % pares, precisión y recall.

Responda: ¿Qué configuración elegiría para un sistema de recomendación en producción? Justifique considerando el trade-off entre costo computacional y calidad de los resultados.

6. Escalando a ml-1m (20 pts)

Ahora aplique las funciones que implementó a un dataset más grande. Reutilice `build_signatures`, `lsh_candidates` y las demás funciones sin modificarlas — solo cambian los datos de entrada.

- 6.1. Cargue ml-1m. Este dataset usa un formato distinto (separador `::`):

```
ratings_1m = pd.read_csv(
    "ruta/ml-1m/ratings.dat",
    sep="::",
    names=["userId", "movieId", "rating", "timestamp"],
    engine="python",
)
```

Imprima: número de usuarios, películas, ratings y total de pares $\binom{n}{2}$.

- 6.2. Construya los conjuntos de películas por usuario, las firmas MinHash y ejecute LSH con los mismos parámetros (t , r) que usó en `ml-latest-small`. Imprima el número de candidatos y la fracción del total de pares.
- 6.3. Calcule la similitud de Jaccard real **solo para los candidatos** (no para todos los pares — serían ~18 millones). Muestre los 10 pares más similares y grafique la distribución de similitudes de los candidatos.

Responda: ¿Qué fracción de los pares tuvo que examinar LSH? ¿Por qué ya no es viable calcular Jaccard exhaustivo a esta escala?

7. Bono: escalando a ml-32m (+10 pts)

Intente ejecutar las funciones implementadas sobre el dataset `ml-32m`.

- 7.1. Cargue ml-32m (`ratings.csv`). Imprima: número de usuarios, películas, ratings y total de pares.
- 7.2. Ejecute cada una de las funciones del pipeline sobre este dataset. ¿Cuáles no terminan en un tiempo razonable (menos de 24 horas)? Use `tqdm` para estimar el tiempo total a partir del progreso parcial.

Responda: ¿Cuál es la función más lenta? ¿Por qué es más lenta que las demás a esta escala?

- 7.3.** Investigue cómo mejorar la implementación de la función más lenta. Proponga e implemente una **versión 2** que sí pueda ejecutarse en tiempo razonable sobre `m1-32m`. Muestre los 10 pares más similares encontrados.

Responda: ¿Qué cambió en su implementación y por qué es más rápida? Reporte el speedup obtenido.